

Section 05 - Vue: Behind The Scenes

Vue Reactivity

The image illustrates the Vue.js reactivity system. On the left, a code editor shows the following JavaScript code:

```
1 const app = Vue.createApp({
2   // ...
3 });
4
5 app.mount("#app");
6
7 const data = {
8   message: "Hello",
9   longMessage: ""
10 };
11
12 const handler = {
13   set(target, key, value) {
14     console.log(target);
15     console.log(key);
16     console.log(value);
17
18     if (key === "message") {
19       target.longMessage = value + " World!";
20     }
21
22     target.message = value;
23   }
24 };
25
26 const proxy = new Proxy(data, handler);
27
28 console.log(proxy);
29
30 proxy.message = "Hi";
31 console.log(proxy);
```

Annotations on the code:

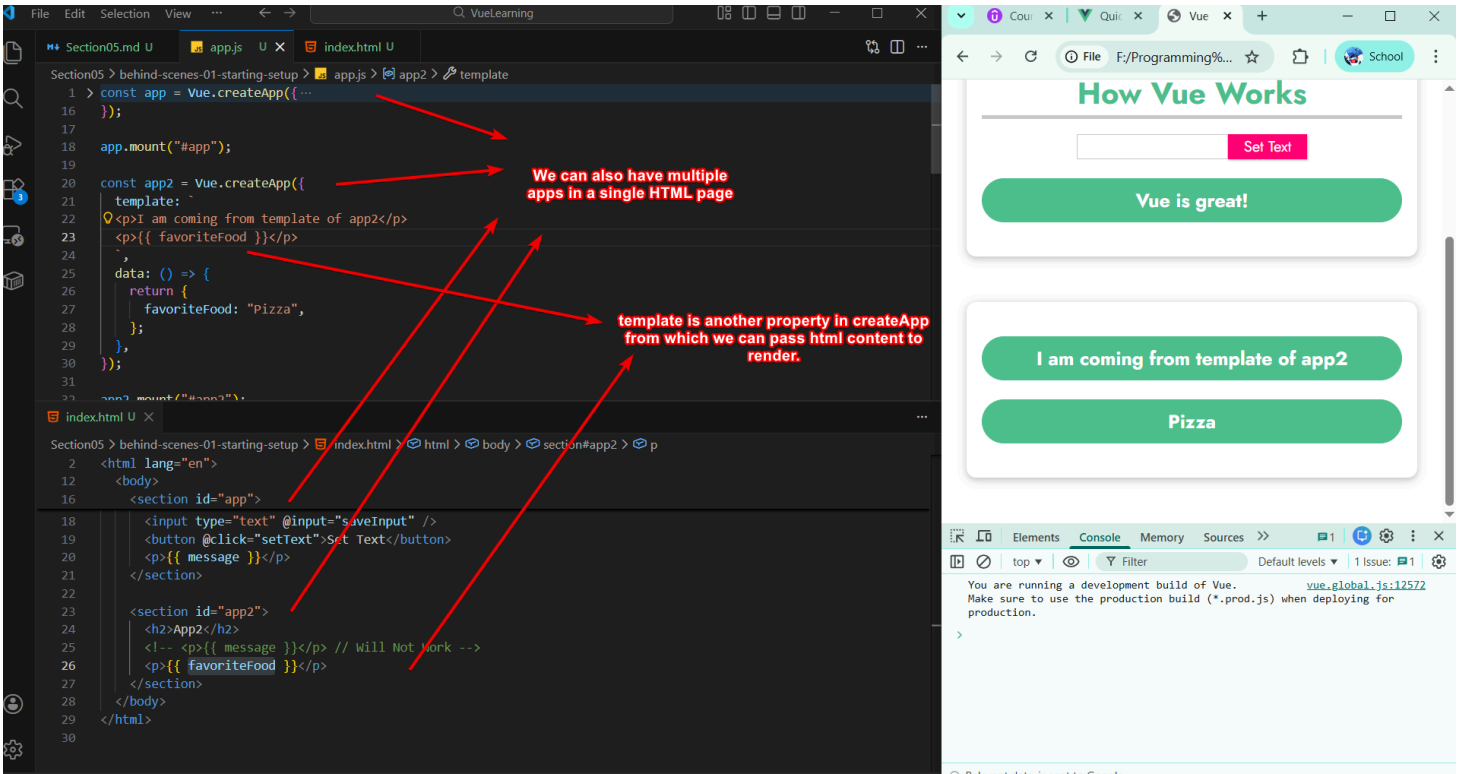
- 1: JavaScript is not reactive by default, Vue Js brings this reactivity using Proxy Object.
- 2: Proxy object accepts data object and handlers object, in the handler object we can pass set function that gets target, key, and value when it is called. So, using it we can update other variable.

On the right, a web browser shows a simple application with a green header "Vue Behind The Scenes", a section titled "How Vue Works", a text input, a "Set Text" button, and a green footer "Vue is great!". The browser's console shows the following log messages:

```
vue.global.js:12572 [Object] {message: 'Hello', LongMessage: ''} app.js:41
{message: 'Hello', LongMessage: ''} app.js:27
message app.js:28
Hi app.js:29
Proxy(Object) {message: 'Hi', LongMessage: 'Hi World!'} app.js:44
```

Vue brings **reactivity** to its functionality using **Proxy** object.

Multiple Apps and Template



We can definitely **have multiple apps** in a single html page and from `template` property we can **pass html content to render**.

Template Ref to Access Element Properties

The screenshot displays a development environment with a code editor and a web browser. The code editor shows the following code:

```
const app = Vue.createApp({
  data() {
    return {
      currentUserInput: "",
      message: "Vue is great!",
    };
  },
  methods: {
    saveInput(event) {
      this.currentUserInput = event.target.value;
    },
    setText() {
      this.message = this.$refs.UserText.value;
    },
  },
});

app.mount("#app");

const app2 = Vue.createApp({
  template: `
    <div>
      <h2>How Vue Works</h2>
      <input type="text" ref="UserText" />
      <button @click="setText">Set Text</button>
      <p>{{ message }}</p>
    </div>
  `,
});
```

Annotations in the code editor:

- Annotation 1: "ref attribute work similar to useRef() in React. We need to provide name to the field and then we can access all properties of an element" (points to the `ref="UserText"` attribute in the HTML template).
- Annotation 2: "VUE JS provides all refs inside an object that can be access via this.\$refs property." (points to the `this.$refs.UserText.value` access in the `setText` method).

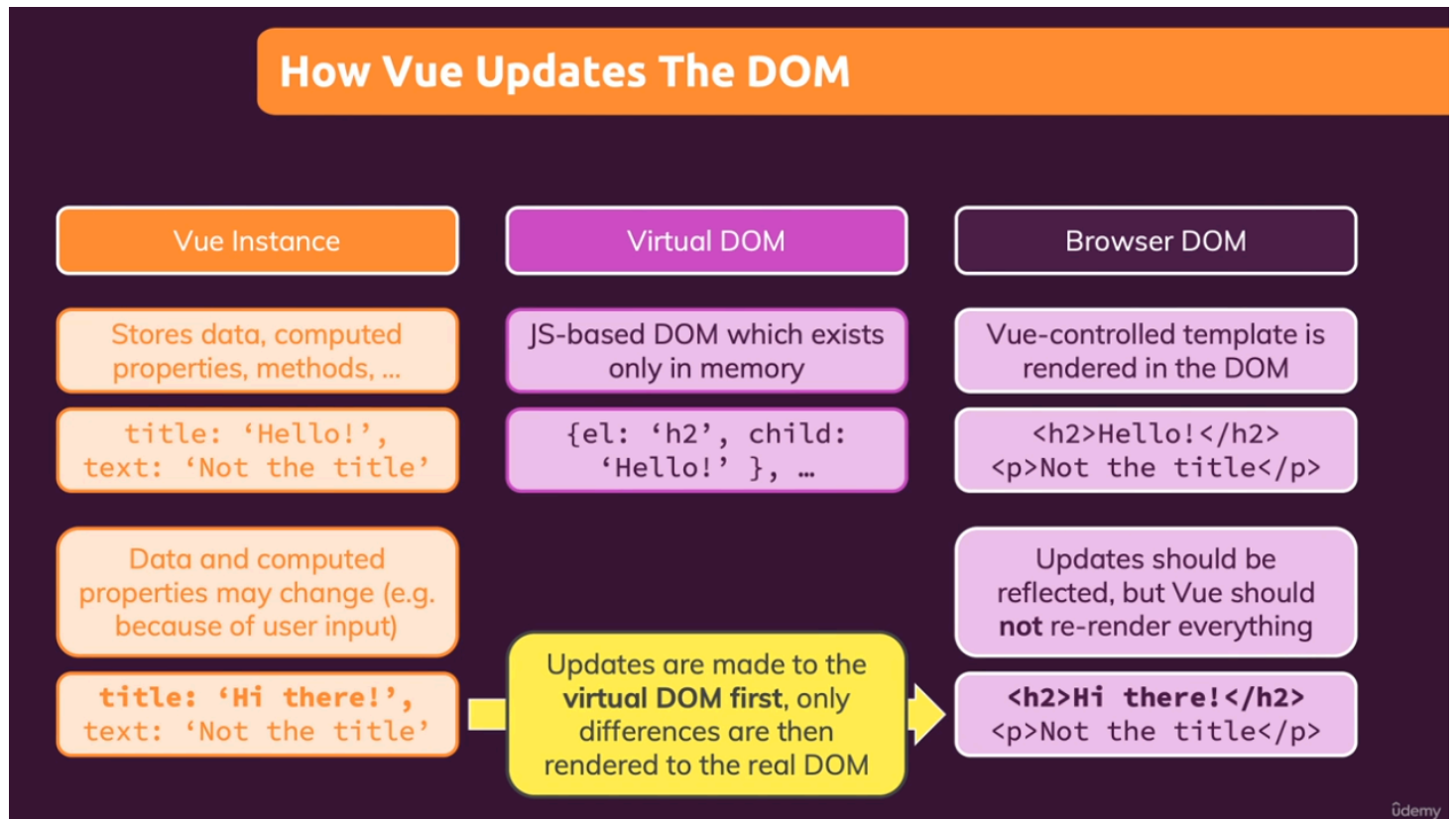
The browser shows the rendered application with the following UI elements:

- A header "How Vue Works".
- A form with an input field containing "does refs work?" and a "Set Text" button.
- A green button labeled "does refs work?".
- A green button labeled "I am coming from template of app2".
- A green button labeled "Pizza".

The browser's console shows a message: "You are running a development build of Vue. Make sure to use the production build (*.prod.js) when deploying for production."

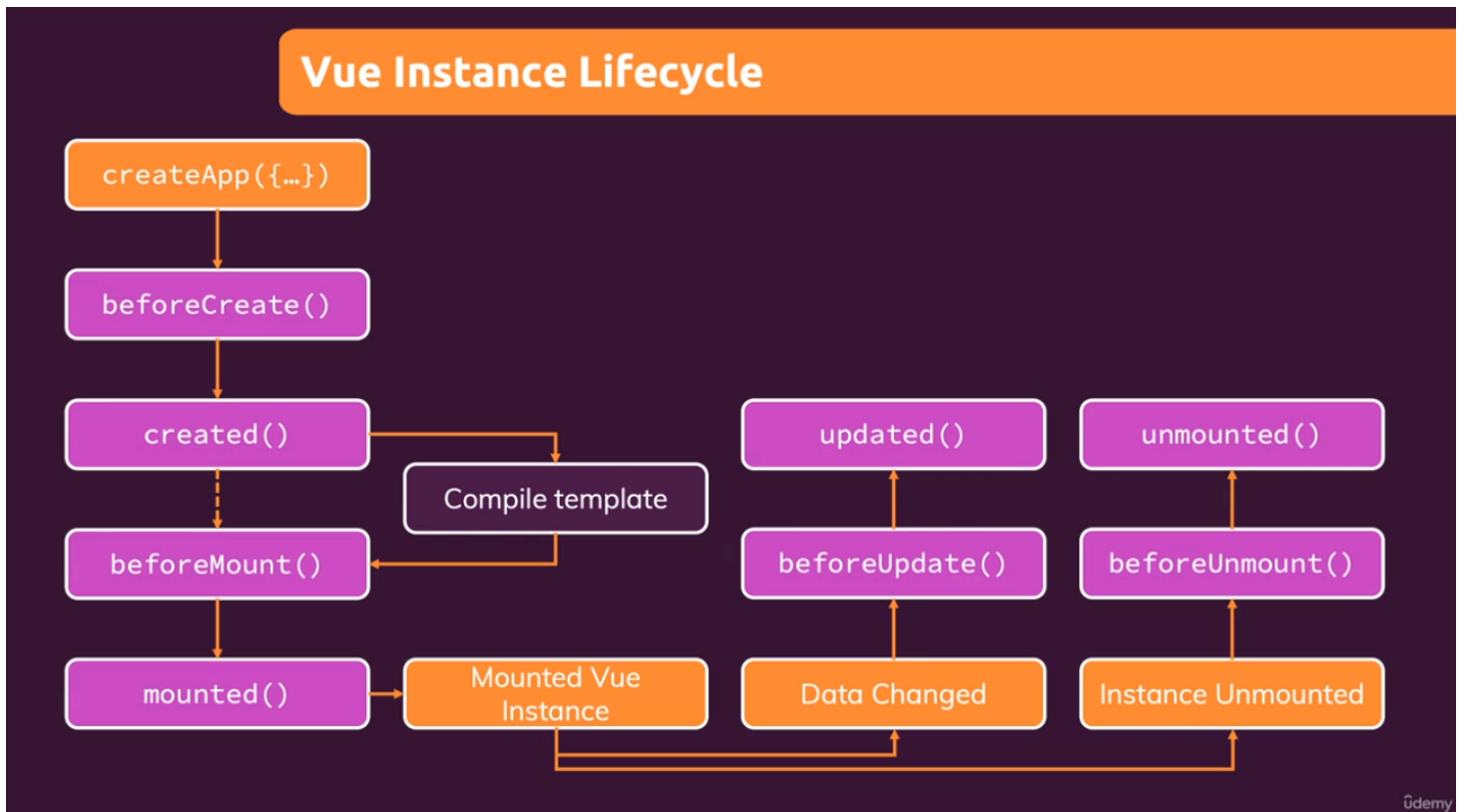
We can access element properties of an element using **ref attribute** and to access these properties Vue JS provides build-in property `$refs` that hold key:value pair for all refs provided in HTML document.

Virtual DOM



Just like React, Vue JS also maintain a copy of **virtual DOM** that it uses to update only those elements that change in actual DOM.

Vue Lifecycle



These are some of the **hooks** that VUE JS provides at **different stages of its lifecycle**. We can use them as property accepting function inside `createApp` method.

```
const app = Vue.createApp({
  data() {
    return {
      currentUserInput: "",
      message: "Vue is great!",
    };
  },

  methods: {
    saveInput(event) {
      this.currentUserInput = event.target.value;
    },
    setText() {
      this.message = this.$refs.UserText.value;
    },
  },

  // this hook will run before app is being created.
  beforeCreate() {
    console.log(`beforeCreated()`);
  },
});
```

Components

Components are **mini-app** that needs **unique identifier** and **config object**. This config object is similar to what we pass to createApp method. So this means components **can have their own data, methods, computed properties, etc.**

File Edit Selection View ...

VueLearning

index.html U app.js U

Section06 > cmp-intro-01-starting-setup > index.html > ...

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8" />
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Vue Basics</title>
7   <link href="https://fonts.googleapis.com/css?family=Roboto" rel="stylesheet">
8   <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
9   <script src="app.js" defer></script>
10 </head>
11 <body>
12 <div id="app">
13   <h1>FriendList</h1>
14   <div id="app">
15     <ul>
16       <li><Friend></Friend></li>
17       <li><Friend></Friend></li>
18       <li><Friend></Friend></li>
19     </ul>
20   </div>
21 </div>
22 </body>
23 </html>
```

Section06 > cmp-intro-01-starting-setup > app.js > ...

```
1 const app = Vue.createApp({
2   data: () => {
3     friends: [
4       {
5         name: "Julie Jones",
6         phone: "09876 543 221",
7         email: "julie@localhost.com",
8       },
9       {
10        name: "Manuel Lorenz",
11        phone: "01234 5678 991",
12        email: "manuel@localhost.com",
13      },
14    ],
15  },
16  methods: {
17    showDetails(friend) {
18      console.log(friend);
19    },
20  },
21  template: `
22    <div>
23      <h2>Manuel Lorenz</h2>
24      <button>Show Details</button>
25      <ul>
26        <li><strong>Phone:</strong> 01234 5678 991</li>
27        <li><strong>Email:</strong> manuel@localhost.com</li>
28      </ul>
29    </div>
30  `,
31  },
32  data: () => {
33    return {
34      friend: {
35        name: "Julie Jones",
36        phone: "09876 543 221",
37        email: "julie@localhost.com",
38      },
39    };
40  },
41 });
42 app.mount("#app");
```

Course: Quick Vu x +

F:/Programm...

School

Email: manuel@localhost.com

Manuel Lorenz

Show Details

Phone: 01234 5678 991

Email: manuel@localhost.com

Manuel Lorenz

Elements Console Memory Sources >>

top Filter Default levels No Issues

You are running a development build of Vue. vue.global.js:12572
Make sure to use the production build (*.prod.js) when deploying for production.

Relevant data is sent to Google

component concept in Vue JS is similar to React, in Vue JS, components are mini-app that needs a unique name and config object similar to what we pass in createApp. So, this mean component can has their own data, method, template, compute, and watchers.